

Guía Completa: Rutas en Laravel 12

Índice

1. [Introducción al Sistema de Rutas](#)
 2. [Rutas Básicas](#)
 3. [Archivos de Rutas](#)
 4. [Parámetros de Ruta](#)
 5. [Rutas Nombradas](#)
 6. [Grupos de Rutas](#)
 7. [Enlace de Modelo \(Model Binding\)](#)
 8. [Rutas de Recursos](#)
 9. [Rate Limiting](#)
 10. [Conceptos Avanzados](#)
-

1. Introducción al Sistema de Rutas

El sistema de rutas en Laravel es el mecanismo fundamental que mapea las peticiones HTTP entrantes a las funciones o métodos específicos de tu aplicación. Es la puerta de entrada a tu aplicación y define qué código se ejecutará cuando un usuario acceda a una URL específica.

¿Qué es una ruta?

Una ruta es básicamente la definición de:

- **Una URI** (por ejemplo: `/usuarios` , `/productos/1`)
- **Un método HTTP** (GET, POST, PUT, DELETE, etc.)
- **Una acción a ejecutar** (una función anónima o un método de controlador)

Ventajas del sistema de rutas de Laravel

- **Sintaxis expresiva y limpia:** Fácil de leer y mantener
 - **Soporte para todos los verbos HTTP:** GET, POST, PUT, PATCH, DELETE, OPTIONS
 - **Parámetros dinámicos:** Captura segmentos de la URL
 - **Middleware integrado:** Protección CSRF, autenticación, etc.
 - **Model Binding:** Inyección automática de modelos Eloquent
 - **Agrupación inteligente:** Comparte atributos entre múltiples rutas
-

2. Rutas Básicas

La estructura más simple

La forma más básica de definir una ruta en Laravel utiliza la facade `Route` con un método HTTP, una URI y una closure (función anónima):

```
use Illuminate\Support\Facades\Route;

Route::get('/saludo', function () {
    return 'Hola Mundo desde Laravel 12';
});
```

Explicación del código:

- `Route::get()` : Define una ruta que responde a peticiones GET
- `'/saludo'` : La URI que activará esta ruta
- `function () { ... }` : La acción a ejecutar (closure)
- `return 'Hola Mundo'` : La respuesta que se enviará al navegador

Métodos HTTP disponibles

Laravel proporciona métodos para cada verbo HTTP:

```
// Peticiones GET (obtener datos)
Route::get('/usuarios', function () {
    return 'Lista de usuarios';
});

// Peticiones POST (crear datos)
Route::post('/usuarios', function () {
    return 'Usuario creado';
});

// Peticiones PUT (actualizar completamente)
Route::put('/usuarios/{id}', function ($id) {
    return "Usuario {$id} actualizado completamente";
});

// Peticiones PATCH (actualizar parcialmente)
Route::patch('/usuarios/{id}', function ($id) {
    return "Usuario {$id} actualizado parcialmente";
});

// Peticiones DELETE (eliminar datos)
Route::delete('/usuarios/{id}', function ($id) {
    return "Usuario {$id} eliminado";
});

// Peticiones OPTIONS (información sobre opciones de comunicación)
Route::options('/usuarios', function () {
    return 'Opciones disponibles';
});
```

Respuestas a múltiples verbos HTTP

A veces necesitas que una ruta responda a varios verbos HTTP:

```
// Responde a GET y POST
Route::match(['get', 'post'], '/formulario', function () {
    return 'Formulario procesado';
});

// Responde a TODOS los verbos HTTP
Route::any('/cualquier-metodo', function () {
```

```
        return 'Acepto cualquier método HTTP';
    });
```

Importante sobre el orden: Cuando defines múltiples rutas con la misma URI, las rutas que usan `get` , `post` , `put` , `patch` , `delete` y `options` deben definirse ANTES que las rutas que usan `any` , `match` o `redirect` .

Rutas de redirección

Para redirecciones simples, Laravel ofrece un atajo:

```
// Redirección con código 302 (temporal)
Route::redirect('/aqui', '/alla');

// Redirección con código personalizado
Route::redirect('/aqui', '/alla', 301);

// Redirección permanente (301)
Route::permanentRedirect('/aqui', '/alla');
```

Rutas de vista directas

Si solo necesitas devolver una vista sin lógica adicional:

```
// Forma básica
Route::view('/bienvenida', 'welcome');

// Pasando datos a la vista
Route::view('/bienvenida', 'welcome', ['nombre' => 'Juan']);
```

Nota: Los parámetros `view` , `data` , `status` y `headers` están reservados por Laravel cuando usas rutas de vista.

Inyección de dependencias en rutas

Laravel automáticamente resuelve e inyecta las dependencias tipadas en tus closures:

```
use Illuminate\Http\Request;

Route::get('/perfil', function (Request $request) {
    // Laravel automáticamente inyecta la petición actual
    $nombre = $request->input('nombre');
    return "Hola, {$nombre}";
});
```

3. Archivos de Rutas

Estructura de archivos de rutas

Laravel organiza las rutas en diferentes archivos dentro del directorio `routes/` :

routes/web.php - Rutas Web

Este archivo es para rutas de tu interfaz web tradicional. Las rutas aquí definidas:

- Se asignan automáticamente al grupo de middleware `web`
- Tienen acceso a sesiones
- Protección CSRF automática
- Gestión de cookies
- Son ideales para aplicaciones que renderizan vistas HTML

```
// routes/web.php
use App\Http\Controllers\UserController;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/usuarios', [UserController::class, 'index']);
```

Para acceder: Simplemente visita `http://tu-dominio.com/usuarios` en tu navegador.

routes/api.php - Rutas API

Para habilitar las rutas API en Laravel 12:

```
php artisan install:api
```

Este comando:

1. Instala Laravel Sanctum (para autenticación API)
2. Crea el archivo `routes/api.php`

Las rutas API:

- Son stateless (sin sesión)
- Se asignan al grupo de middleware `api`
- Tienen el prefijo `/api` automáticamente
- Incluyen rate limiting

```
// routes/api.php
use Illuminate\Http\Request;

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');
```

Para acceder: `http://tu-dominio.com/api/user`

Personalizar el prefijo de API

Puedes cambiar el prefijo `/api` por defecto en `bootstrap/app.php` :

```

->withRouting(
    api: __DIR__.'../routes/api.php',
    apiPrefix: 'api/v1', // Ahora será /api/v1/
    // ...
)

```

Archivos de rutas personalizados

Puedes crear archivos de rutas adicionales usando el método `then` en `bootstrap/app.php` :

```

use Illuminate\Support\Facades\Route;

->withRouting(
    web: __DIR__.'../routes/web.php',
    api: __DIR__.'../routes/api.php',
    commands: __DIR__.'../routes/console.php',
    health: '/up',
    then: function () {
        // Archivo de rutas para webhooks
        Route::middleware('api')
            ->prefix('webhooks')
            ->name('webhooks.')
            ->group(base_path('routes/webhooks.php'));

        // Archivo de rutas para admin
        Route::middleware(['web', 'auth', 'admin'])
            ->prefix('admin')
            ->name('admin.')
            ->group(base_path('routes/admin.php'));
    },
)

```

Control total sobre el registro de rutas

Para controlar completamente cómo se cargan las rutas:

```

->withRouting(
    commands: __DIR__.'../routes/console.php',
    using: function () {
        Route::middleware('api')
            ->prefix('api')
            ->group(base_path('routes/api.php'));

        Route::middleware('web')
            ->group(base_path('routes/web.php'));
    },
)

```

Listar todas las rutas

Laravel proporciona un comando Artisan muy útil:

```
# Listar todas las rutas
php artisan route:list

# Mostrar middleware asignado
php artisan route:list -v

# Mostrar grupos de middleware expandidos
php artisan route:list -vv

# Filtrar por URI
php artisan route:list --path=api

# Filtrar por método HTTP
php artisan route:list --method=GET

# Filtrar por nombre de ruta
php artisan route:list --name=users

# Excluir rutas de paquetes externos
php artisan route:list --except=vendor

# Mostrar solo rutas de paquetes
php artisan route:list --only=vendor

# Ordenar por URI
php artisan route:list --sort=uri

# Ordenar inversamente
php artisan route:list --reverse
```

4. Parámetros de Ruta

Los parámetros de ruta te permiten capturar segmentos dinámicos de la URI.

Parámetros requeridos

```
// Parámetro simple
Route::get('/usuario/{id}', function (string $id) {
    return "Perfil del usuario: {$id}";
});

// Múltiples parámetros
Route::get('/posts/{post}/comentarios/{comentario}', function (string $postId,
string $comentarioId) {
    return "Post {$postId}, Comentario {$comentarioId}";
});
```

Reglas para parámetros:

- Se encierran entre llaves {}
- Deben contener caracteres alfabéticos
- Se permiten guiones bajos _
- Se inyectan en orden (los nombres de las variables no importan)

Parámetros con inyección de dependencias

```
use Illuminate\Http\Request;

Route::get('/usuario/{id}', function (Request $request, string $id) {
    // Las dependencias van primero, los parámetros después
    $email = $request->input('email');
    return "Usuario {$id}, Email: {$email}";
});
```

Parámetros opcionales

Marca un parámetro como opcional con ? y proporciona un valor por defecto:

```
// Parámetro opcional con valor null por defecto
Route::get('/usuario/{nombre?}', function (?string $nombre = null) {
    return $nombre ?? 'Invitado';
});

// Parámetro opcional con valor específico por defecto
Route::get('/usuario/{nombre?}', function (string $nombre = 'Juan') {
    return "Hola, {$nombre}";
});
```

Restricciones con expresiones regulares

Puedes validar el formato de los parámetros usando el método where :

```
// Solo números
Route::get('/usuario/{id}', function (string $id) {
    return "Usuario ID: {$id}";
})->where('id', '[0-9]+');

// Solo letras
Route::get('/usuario/{nombre}', function (string $nombre) {
    return "Usuario: {$nombre}";
})->where('nombre', '[A-Za-z]+');

// Múltiples restricciones
Route::get('/usuario/{id}/{nombre}', function (string $id, string $nombre) {
    return "ID: {$id}, Nombre: {$nombre}";
})->where([
    'id' => '[0-9]+',
    'nombre' => '[a-z]+'
]);
```

Métodos de restricción predefinidos

Laravel ofrece métodos helper para patrones comunes:

```
// Solo números
Route::get('/usuario/{id}', function (string $id) {
    //...
})->whereNumber('id');

// Solo letras
Route::get('/usuario/{nombre}', function (string $nombre) {
    //...
})->whereAlpha('nombre');

// Solo alfanumérico
Route::get('/usuario/{username}', function (string $username) {
    //...
})->whereAlphaNumeric('username');

// UUID válido
Route::get('/usuario/{uuid}', function (string $uuid) {
    //...
})->whereUuid('uuid');

// ULID válido
Route::get('/usuario/{ulid}', function (string $ulid) {
    //...
})->whereUlid('ulid');

// Solo valores específicos
Route::get('/categoria/{categoria}', function (string $categoria) {
    //...
})->whereIn('categoria', ['pelicula', 'musica', 'arte']);

// Usando enums
use App\Enums\CategoryEnum;

Route::get('/categoria/{categoria}', function (string $categoria) {
    //...
})->whereIn('categoria', CategoryEnum::cases());

// Encadenar múltiples restricciones
Route::get('/usuario/{id}/{nombre}', function (string $id, string $nombre) {
    //...
})->whereNumber('id')->whereAlpha('nombre');
```

Restricciones globales

Para aplicar restricciones a todos los parámetros con cierto nombre, usa `Route::pattern()` en el método `boot` de `App\Providers\AppServiceProvider` :

```

use Illuminate\Support\Facades\Route;

public function boot(): void
{
    // Ahora TODOS los parámetros {id} en TODAS las rutas
    // solo aceptarán números
    Route::pattern('id', '[0-9]+');

    // Múltiples patrones globales
    Route::pattern('slug', '[a-z0-9-]+');
    Route::pattern('uuid', '[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}');
}

```

Barras inclinadas codificadas

Por defecto, Laravel no permite `/` en parámetros. Para permitirlo:

```

Route::get('/buscar/{busqueda}', function (string $busqueda) {
    return "Buscando: {$busqueda}";
})->where('busqueda', '.*'); // Permite cualquier carácter, incluyendo /

```

Nota: Las barras codificadas solo funcionan en el último segmento de la ruta.

5. Rutas Nombradas

Las rutas nombradas permiten generar URLs y redirecciones de forma conveniente sin hardcodear las URLs.

Asignar nombres a rutas

```

// Con closure
Route::get('/perfil/usuario', function () {
    //...
})->name('perfil');

// Con controlador
Route::get('/perfil/usuario', [UserProfileController::class, 'show'])
    ->name('perfil');

// Con múltiples métodos
Route::get('/usuario/{id}/editar', [UserController::class, 'edit'])
    ->name('usuario.editar')
    ->middleware('auth');

```

Importante: Los nombres de rutas deben ser únicos en toda la aplicación.

Generar URLs desde rutas nombradas

```
// Generar URL
$url = route('perfil');
// Resultado: http://tu-dominio.com/perfil/usuario

// Generar URL con parámetros
$url = route('usuario.editar', ['id' => 1]);
// Resultado: http://tu-dominio.com/usuario/1/editar

// Parámetros adicionales se agregan como query string
$url = route('perfil', ['id' => 1, 'fotos' => 'si']);
// Resultado: http://tu-dominio.com/perfil/usuario?id=1&fotos=si
```

Generar redirecciones

```
// Redirigir a ruta nombrada
return redirect()->route('perfil');

// Redirigir con parámetros
return redirect()->route('usuario.editar', ['id' => 1]);

// Atajo con to_route()
return to_route('perfil');
return to_route('usuario.editar', ['id' => 1]);
```

Uso en vistas Blade

```
<!-- Enlace simple -->
<a href="{ route('perfil') }}">Mi Perfil</a>

<!-- Enlace con parámetros -->
<a href="{ route('usuario.editar', ['id' => $usuario->id]) }}">
    Editar Usuario
</a>

<!-- Formulario -->
<form action="{ route('usuario.actualizar', ['id' => $usuario->id]) }"
method="POST">
    @csrf
    @method('PUT')
    <!-- campos del formulario -->
</form>
```

Inspeccionar la ruta actual

Desde un middleware o cualquier lugar con acceso a la petición:

```
use Illuminate\Http\Request;
```

```

public function handle(Request $request, Closure $next)
{
    // Verificar si la ruta actual tiene un nombre específico
    if ($request->route()->named('perfil')) {
        // Hacer algo especial
    }

    // Verificar múltiples nombres
    if ($request->route()->named(['perfil', 'dashboard'])) {
        //...
    }

    // Obtener el nombre de la ruta actual
    $nombreRuta = $request->route()->getName();

    return $next($request);
}

```

Valores por defecto para parámetros

Puedes establecer valores predeterminados globales para parámetros de URL:

```

use Illuminate\Support\Facades\URL;

// En AppServiceProvider@boot
URL::defaults(['locale' => 'es']);

// Ahora todas las URLs generadas incluirán locale=es automáticamente

```

6. Grupos de Rutas

Los grupos permiten compartir atributos entre múltiples rutas sin repetir código.

Aplicar middleware a grupos

```

Route::middleware(['auth', 'verificado'])->group(function () {
    Route::get('/dashboard', function () {
        // Requiere autenticación y verificación
    });

    Route::get('/perfil', function () {
        // También requiere autenticación y verificación
    });
});

// Middleware múltiple en orden
Route::middleware(['primero', 'segundo', 'tercero'])->group(function () {
    // Se ejecutan en orden: primero → segundo → tercero
});

```

Agrupar rutas por controlador

```
use App\Http\Controllers\OrderController;

Route::controller(OrderController::class)->group(function () {
    Route::get('/ordenes/{id}', 'show');      // OrderController@show
    Route::post('/ordenes', 'store');         // OrderController@store
    Route::put('/ordenes/{id}', 'update');    // OrderController@update
    Route::delete('/ordenes/{id}', 'destroy'); // OrderController@destroy
});
```

Prefijos de URI

```
// Todas las rutas comenzarán con /admin
Route::prefix('admin')->group(function () {
    Route::get('/usuarios', function () {
        // Coincide con /admin/usuarios
    });

    Route::get('/productos', function () {
        // Coincide con /admin/productos
    });
});
```

Prefijos de nombres de ruta

```
// Todos los nombres de ruta comenzarán con "admin."
Route::name('admin.')->group(function () {
    Route::get('/usuarios', function () {
        // Nombre: admin.usuarios
    })->name('usuarios');

    Route::get('/productos', function () {
        // Nombre: admin.productos
    })->name('productos');
});
```

Combinando atributos de grupo

```
Route::middleware(['auth', 'admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {
        Route::get('/usuarios', [UserController::class, 'index'])
            ->name('usuarios.index');
        // URI: /admin/usuarios
        // Nombre: admin.usuarios.index
    });
```

```

        // Middleware: auth, admin

Route::get('/usuarios/{id}', [UserController::class, 'show'])
    ->name('usuarios.show');
// URI: /admin/usuarios/{id}
// Nombre: admin.usuarios.show
});

```

Enrutamiento de subdominios

```

// Capturar subdominios dinámicamente
Route::domain('{cuenta}.miapp.com')->group(function () {
    Route::get('/usuario/{id}', function (string $cuenta, string $id) {
        // $cuenta contendrá el subdominio (ej: "empresa1")
        // $id contendrá el ID del usuario
        return "Cuenta: {$cuenta}, Usuario: {$id}";
    });
});

// Ejemplo real: Sistema multitenancy
Route::domain('{tenant}.acmeapp.com')->group(function () {
    Route::get('/', function ($tenant) {
        $account = Account::where('subdomain', $tenant)->firstOrFail();
        return view('tenant.dashboard', compact('account'));
    });

    Route::get('/facturacion', [BillingController::class, 'index']);
    Route::get('/configuracion', [SettingsController::class, 'index']);
});

```

Importante: Registra las rutas de subdominios ANTES de las rutas de dominio raíz para evitar que sean sobrescritas.

Grupos anidados

Los grupos se pueden anidar y sus atributos se combinan inteligentemente:

```

Route::prefix('admin')->name('admin.')->group(function () {
    // Grupo nivel 1: /admin/*, admin.*

    Route::prefix('usuarios')->name('usuarios.')->group(function () {
        // Grupo nivel 2: /admin/usuarios/*, admin.usuarios.*

        Route::get('/', [UserController::class, 'index'])
            ->name('index');
        // URI: /admin/usuarios/
        // Nombre: admin.usuarios.index

        Route::get('/{id}', [UserController::class, 'show'])
            ->name('show');
    });
});

```

```
        // URI: /admin/usuarios/{id}
        // Nombre: admin.usuarios.show
    });
});
```

7. Enlace de Modelo (Model Binding)

El Model Binding inyecta automáticamente instancias de modelos Eloquent en tus rutas.

Enlace implícito básico

```
use App\Models\User;

Route::get('/usuarios/{user}', function (User $user) {
    // Laravel busca automáticamente User::find($id)
    // Si no existe, devuelve 404
    return $user->email;
});
```

¿Cómo funciona?

1. Laravel ve el parámetro `{user}` en la URI
2. Ve el type-hint `User $user` en la función
3. Automáticamente ejecuta `User::find($valor_del_parametro)`
4. Inyecta el modelo encontrado o retorna 404

Enlace implícito en controladores

```
// Ruta
Route::get('/usuarios/{user}', [UserController::class, 'show']);

// Controlador
public function show(User $user)
{
    // $user ya contiene el modelo completo
    return view('user.profile', ['user' => $user]);
}
```

Personalizar la columna de búsqueda

Por defecto, Laravel busca por `id`, pero puedes especificar otra columna:

```
use App\Models\Post;

// Buscar por slug en lugar de id
Route::get('/posts/{post:slug}', function (Post $post) {
    // Laravel ejecuta: Post::where('slug', $valor)->firstOrFail()
    return $post;
});
```

```
// Ejemplo de uso:
// URL: /posts/mi-primer-post
// Laravel busca: Post donde slug = 'mi-primer-post'
```

Cambiar la columna por defecto globalmente

En tu modelo Eloquent:

```
class Post extends Model
{
    /**
     * Obtener la clave de ruta para el modelo
     */
    public function getRouteKeyName(): string
    {
        return 'slug'; // Ahora siempre buscará por slug
    }
}

// Ahora puedes usar:
Route::get('/posts/{post}', function (Post $post) {
    // Busca por slug automáticamente
});
```

Scoping de relaciones

Cuando tienes rutas anidadas, Laravel puede scope automáticamente:

```
use App\Models\User;
use App\Models\Post;

Route::get('/usuarios/{user}/posts/{post:slug}', function (User $user, Post $post) {
    // Laravel automáticamente verifica que el post pertenezca al user
    // Usando la relación: $user->posts()->where('slug', $post)->first()
    return $post;
});
```

Cómo funciona:

- Laravel asume que `User` tiene una relación `posts()`
- Busca el post dentro de esa relación
- Si el post no pertenece al usuario, retorna 404

Scoping explícito

Para activar scoping cuando NO usas claves personalizadas:

```
Route::get('/usuarios/{user}/posts/{post}', function (User $user, Post $post) {
    return $post;
})->scopeBindings(); // Activa el scoping explícitamente
```

Scoping a nivel de grupo

```
Route::scopeBindings()->group(function () {
    Route::get('/usuarios/{user}/posts/{post}', function (User $user, Post $post) {
        return $post;
    });

    // Todas las rutas en este grupo tendrán scoping
});
```

Desactivar scoping

```
Route::get('/usuarios/{user}/posts/{post:slug}', function (User $user, Post $post) {
    // El post NO necesita pertenecer al usuario
    return $post;
})->withoutScopedBindings();
```

Modelos eliminados suavemente (Soft Deletes)

Por defecto, el model binding no recupera modelos eliminados suavemente:

```
use App\Models\User;

// Incluir modelos eliminados
Route::get('/usuarios/{user}', function (User $user) {
    return $user->email;
})->withTrashed();
```

Personalizar comportamiento cuando falta el modelo

```
use App\Http\Controllers\LocationsController;
use Illuminate\Http\Request;

Route::get('/ubicaciones/{location:slug}', [LocationsController::class, 'show'])
->name('locations.view')
->missing(function (Request $request) {
    // En lugar de 404, redirigir a lista
    return redirect()->route('locations.index');
});
```

Enlace implícito con Enums

Laravel 12 soporta enums como parámetros:

```
// Enum
enum Category: string
{
    case Fruits = 'fruits';
```

```

        case People = 'people';
    }

    // Ruta
    Route::get('/categorias/{category}', function (Category $category) {
        return $category->value;
    });

    // Solo acepta: /categorias/fruits o /categorias/people
    // Cualquier otro valor retorna 404

```

Enlace explícito

Para control total sobre cómo se resuelven los modelos:

```

use App\Models\User;
use Illuminate\Support\Facades\Route;

// En AppServiceProvider@boot
public function boot(): void
{
    // Enlace explícito básico
    Route::model('user', User::class);

    // Enlace con lógica personalizada
    Route::bind('user', function (string $value) {
        // Buscar por nombre en lugar de ID
        return User::where('name', $value)->firstOrFail();
    });
}

```

Personalizar resolución en el modelo

```

class User extends Model
{
    /**
     * Resolver el modelo para un valor dado
     */
    public function resolveRouteBinding($value, $field = null)
    {
        // Lógica personalizada
        return $this->where('username', $value)
            ->where('activo', true)
            ->firstOrFail();
    }

    /**
     * Resolver enlaces hijos (para scoping)
     */
    public function resolveChildRouteBinding($childType, $value, $field)
    {
    }
}

```

```

    {
      // Lógica para relaciones anidadas
      return parent::resolveChildRouteBinding($childType, $value, $field);
    }
  }
}

```

Múltiples recursos

```

Route::resources([
  'photos' => PhotoController::class,
  'posts' => PostController::class,
  'videos' => VideoController::class,
]);

```

Recursos parciales

Si no necesitas todas las rutas:

```

// Solo crear rutas específicas
Route::resource('photos', PhotoController::class)
  ->only(['index', 'show']);

// Crear todas EXCEPTO las especificadas
Route::resource('photos', PhotoController::class)
  ->except(['create', 'store', 'destroy']);

```

Rutas API de recursos

Para APIs, no necesitas las rutas `create` y `edit` (que muestran formularios):

```

Route::apiResource('photos', PhotoController::class);

```

Esto crea solo 5 rutas:

Verbo HTTP	URI	Acción	Nombre de Ruta
GET	/photos	index	photos.index
POST	/photos	store	photos.store
GET	/photos/{photo}	show	photos.show
PUT/PATCH	/photos/{photo}	update	photos.update
DELETE	/photos/{photo}	destroy	photos.destroy

Múltiples API resources

```

Route::apiResources([
  'photos' => PhotoController::class,

```

```
'posts' => PostController::class,
]);
```

Recursos anidados

```
// Fotos dentro de álbumes
Route::resource('albums.photos', PhotoController::class);
```

Esto crea rutas como:

- GET /albums/{album}/photos - albums.photos.index
- POST /albums/{album}/photos - albums.photos.store
- GET /albums/{album}/photos/{photo} - albums.photos.show

El controlador recibe ambos parámetros:

```
public function index(Album $album)
{
    return $album->photos; // Scoped automáticamente
}

public function show(Album $album, Photo $photo)
{
    // $photo pertenece a $album
    return view('photos.show', compact('album', 'photo'));
}
```

Anidamiento superficial (Shallow Nesting)

Para evitar URIs muy largas en rutas anidadas:

```
Route::resource('albums.photos', PhotoController::class)->shallow();
```

Esto crea:

Verbo	URI	Acción	Nombre de Ruta
GET	/albums/{album}/photos	index	albums.photos.index
POST	/albums/{album}/photos	store	albums.photos.store
GET	/photos/{photo}	show	photos.show
PUT	/photos/{photo}	update	photos.update
DELETE	/photos/{photo}	destroy	photos.destroy

Personalizar nombres de parámetros

```
Route::resource('users', UserController::class)->parameters([
    'users' => 'persona'
]);

// Ahora las rutas usan {persona} en lugar de {user}
// GET /users/{persona}
```

Personalizar nombres de rutas

```
Route::resource('photos', PhotoController::class)->names([
    'create' => 'photos.build',
    'store' => 'photos.save',
]);

// Ahora puedes usar:
// route('photos.build') en lugar de route('photos.create')
```

Recursos singleton

Para recursos que siempre son únicos (como perfil del usuario):

```
Route::singleton('profile', ProfileController::class);
```

Esto NO incluye rutas con {id} :

Verbo HTTP	URI	Acción	Nombre de Ruta
GET	/profile	show	profile.show
GET	/profile/edit	edit	profile.edit
PUT/PATCH	/profile	update	profile.update

9. Rate Limiting (Limitación de Tasa)

El rate limiting te permite controlar cuántas veces un usuario puede acceder a ciertas rutas en un período de tiempo.

Definir limitadores de tasa

En `App\Providers\AppServiceProvider` :

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\RateLimiter;

public function boot(): void
{
    // Limitar a 60 peticiones por minuto
    RateLimiter::for('api', function (Request $request) {
```

```

        return Limit::perMinute(60)
            ->by($request->user()?->id ?: $request->ip());
    });

    // Límite global
    RateLimiter::for('global', function (Request $request) {
        return Limit::perMinute(1000);
    });
}

```

Métodos de límite disponibles

```

// Por minuto
Limit::perMinute(100)

// Por hora
Limit::perHour(1000)

// Por día
Limit::perDay(10000)

// Por segundos
Limit::perSecond(10)

// Personalizado (por 5 minutos)
Limit::perMinutes(5, 50)

// Sin límite (usuarios VIP)
Limit::none()

```

Segmentar límites

```

RateLimiter::for('api', function (Request $request) {
    return $request->user()
        ? Limit::perMinute(100)->by($request->user()->id)
        : Limit::perMinute(10)->by($request->ip());
});

// Usuarios VIP sin límite
RateLimiter::for('uploads', function (Request $request) {
    return $request->user()->vipCustomer()
        ? Limit::none()
        : Limit::perHour(10)->by($request->user()->id);
});

```

Múltiples límites

```
RateLimiter::for('login', function (Request $request) {
    return [
        // Límite global por minuto
        Limit::perMinute(500),

        // Límite por email (previene ataques de fuerza bruta)
        Limit::perMinute(3)->by($request->input('email')),
    ];
});
```

Importante: Cuando uses múltiples límites con el mismo `by()`, asegúrate de que sean únicos:

```
RateLimiter::for('uploads', function (Request $request) {
    return [
        Limit::perMinute(10)->by('minute:'.$request->user()->id),
        Limit::perDay(1000)->by('day:'.$request->user()->id),
    ];
});
```

Rate limiting basado en respuestas

Cuenta solo ciertas respuestas hacia el límite:

```
use Symfony\Component\HttpFoundation\Response;

RateLimiter::for('resource-not-found', function (Request $request) {
    return Limit::perMinute(10)
        ->by($request->user()?->id ?: $request->ip())
        ->after(function (Response $response) {
            // Solo cuenta respuestas 404
            return $response->status() === 404;
        });
});

// Ejemplo: No contar errores de validación
RateLimiter::for('create-post', function (Request $request) {
    return Limit::perMinute(5)
        ->by($request->user()->id)
        ->after(function (Response $response) {
            // Solo cuenta operaciones exitosas
            return $response->status() === 201;
        });
});
```

Personalizar respuesta cuando se excede el límite

```
RateLimiter::for('api', function (Request $request) {
    return Limit::perMinute(60)->response(function (Request $request, array $headers) {
```

```

        return response()->json([
            'message' => 'Demasiadas peticiones. Por favor espera.',
            'retry_after' => $headers['Retry-After'] ?? 60,
        ], 429, $headers);
    });
});

```

Aplicar rate limiting a rutas

```

// A una ruta específica
Route::middleware(['throttle:api'])->group(function () {
    Route::post('/audio', function () {
        //...
    });
});

// A un grupo de rutas
Route::middleware(['throttle:uploads'])->group(function () {
    Route::post('/audio', function () {
        //...
    });

    Route::post('/video', function () {
        //...
    });
});

// A ruta individual
Route::post('/api/users', [UserController::class, 'store'])
    ->middleware('throttle:api');

```

Rate limiting con Redis

Si usas Redis como cache driver, puedes usar una implementación más eficiente:

En `bootstrap/app.php` :

```

->withMiddleware(function (Middleware $middleware) {
    $middleware->throttleWithRedis();
})

```

Headers de respuesta

Cuando se aplica rate limiting, Laravel incluye automáticamente estos headers:

X-RateLimit-Limit: 60	# Límite total
X-RateLimit-Remaining: 59	# Peticiones restantes
Retry-After: 60	# Segundos hasta reinicio (solo cuando se excede)

Ejemplo práctico completo

```
// AppServiceProvider@boot
public function boot(): void
{
    // API pública - límite estricto
    RateLimiter::for('api-public', function (Request $request) {
        return Limit::perMinute(30)->by($request->ip());
    });

    // API autenticada - límite generoso
    RateLimiter::for('api-auth', function (Request $request) {
        return Limit::perMinute(100)->by($request->user()->id);
    });

    // Login - prevenir brute force
    RateLimiter::for('login', function (Request $request) {
        return [
            Limit::perMinute(5)->by($request->input('email')),
            Limit::perHour(20)->by($request->ip()),
        ];
    });

    // Uploads - límite por tamaño de plan
    RateLimiter::for('uploads', function (Request $request) {
        $user = $request->user();

        return match($user->plan) {
            'free' => Limit::perDay(10),
            'pro' => Limit::perDay(100),
            'enterprise' => Limit::none(),
        };
    });
}

// Aplicar en rutas
Route::post('/login', [AuthController::class, 'login'])
    ->middleware('throttle:login');

Route::middleware(['auth:sanctum', 'throttle:api-auth'])->group(function () {
    Route::get('/user', function (Request $request) {
        return $request->user();
    });

    Route::post('/posts', [PostController::class, 'store'])
        ->middleware('throttle:uploads');
});
```

10. Conceptos Avanzados

Rutas Fallback

Una ruta que captura todas las peticiones no coincidentes:

```
Route::fallback(function () {
    return response()->json([
        'message' => 'Ruta no encontrada'
    ], 404);
});

// Con vista personalizada
Route::fallback(function () {
    return view('errors.404');
});
```

Importante: La ruta fallback debe ser la última ruta registrada.

Protección CSRF

Las rutas POST, PUT, PATCH y DELETE requieren token CSRF:

```
<form method="POST" action="/profile">
    @csrf
    <!-- campos del formulario -->
</form>
```

Para rutas PUT, PATCH, DELETE desde formularios HTML:

```
<form method="POST" action="/usuario/1">
    @csrf
    @method('PUT')
    <!-- campos -->
</form>

<!-- 0 manualmente -->
<form method="POST" action="/usuario/1">
    <input type="hidden" name="_token" value="{{ csrf_token() }}">
    <input type="hidden" name="_method" value="PUT">
    <!-- campos -->
</form>
```

Spoofing de métodos HTTP

HTML solo soporta GET y POST, pero puedes simular otros métodos:

```
<!-- DELETE -->
<form action="/usuario/1" method="POST">
    @method('DELETE')
    @csrf
    <button type="submit">Eliminar</button>
</form>
```

```

<!-- PUT -->
<form action="/usuario/1" method="POST">
    @method('PUT')
    @csrf
    <!-- campos -->
</form>

<!-- PATCH -->
<form action="/usuario/1" method="POST">
    @method('PATCH')
    @csrf
    <!-- campos -->
</form>

```

Acceder a información de la ruta actual

```

use Illuminate\Support\Facades\Route;

// Obtener instancia de la ruta actual
$route = Route::current();

// Obtener nombre de la ruta actual
$name = Route::currentRouteName();

// Obtener acción de la ruta actual
$action = Route::currentRouteAction();

// Desde Request
$route = $request->route();
$name = $request->route()->getName();

// Verificar si es una ruta específica
if ($request->route()->named('profile')) {
    //...
}

// Verificar múltiples nombres
if ($request->route()->named(['profile', 'dashboard'])) {
    //...
}

// Obtener parámetros de la ruta
$id = $request->route('id');
$params = $request->route()->parameters();

```

CORS (Cross-Origin Resource Sharing)

Laravel maneja CORS automáticamente. Para personalizar:

```
php artisan config:publish cors
```

Esto crea `config/cors.php` :

```
return [
    'paths' => ['api/*', 'sanctum/csrf-cookie'],

    'allowed_methods' => ['*'],

    'allowed_origins' => ['*'],

    'allowed_origins_patterns' => [],

    'allowed_headers' => ['*'],

    'exposed_headers' => [],

    'max_age' => 0,

    'supports_credentials' => false,
];
```

Ejemplo de configuración restrictiva:

```
return [
    'paths' => ['api/*'],

    'allowed_methods' => ['GET', 'POST', 'PUT', 'DELETE'],

    'allowed_origins' => [
        'https://miapp.com',
        'https://www.miapp.com',
    ],

    'allowed_headers' => ['Content-Type', 'Authorization'],

    'exposed_headers' => ['X-Total-Count'],

    'max_age' => 3600,

    'supports_credentials' => true,
];
```

Caché de rutas

En producción, cachea tus rutas para mejorar el rendimiento:

```
# Crear caché de rutas
php artisan route:cache
```

```
# Limpiar caché de rutas
php artisan route:clear
```

Importante:

- No puedes usar closures en rutas si usas caché
- Debes usar solo controladores
- Regenera el caché cada vez que cambies rutas

```
// NO funcionará con cache
Route::get('/', function () {
    return view('welcome');
});

// SÍ funcionará con cache
Route::get('/', [HomeController::class, 'index']);
```

Macros de ruta personalizadas

Puedes extender el router con tus propios métodos:

```
// En AppServiceProvider@boot
use Illuminate\Support\Facades\Route;

Route::macro('softDeletes', function () {
    Route::get($this->uri.'/{id}/restore', $this->action.'@restore')
        ->name($this->name.'.restore');

    Route::get($this->uri.'/{trashed}', $this->action.'@trashed')
        ->name($this->name.'.trashed');

    return $this;
});

// Uso
Route::resource('posts', PostController::class)->softDeletes();
```

Middleware en línea

Puedes definir middleware directamente en la ruta:

```
Route::get('/profile', function () {
    //...
})->middleware(function ($request, $next) {
    if ($request->user()->isAdmin()) {
        return $next($request);
    }

    return redirect('/');
});
```

Rutas condicionales

```
// Solo registrar en entornos específicos
if (app()->environment('local', 'staging')) {
    Route::get('/debug', [DebugController::class, 'index']);
}

// Basado en configuración
if (config('app.enable_api')) {
    Route::prefix('api')->group(function () {
        // rutas API
    });
}
```

Subdominios dinámicos con constraints

```
Route::domain('{account}.myapp.com')
    ->where('account', '[a-z0-9-]+')
    ->group(function () {
        Route::get('/', function ($account) {
            //...
        });
    });
```

Generación de URLs firmadas

URLs que expiran o están firmadas para seguridad:

```
use Illuminate\Support\Facades\URL;

// Generar URL firmada
$url = URL::signedRoute('unsubscribe', ['user' => 1]);

// URL que expira en 30 minutos
$url = URL::temporarySignedRoute(
    'unsubscribe',
    now()->addMinutes(30),
    ['user' => 1]
);

// En la ruta
Route::get('/unsubscribe/{user}', function (Request $request) {
    if (!$request->hasValidSignature()) {
        abort(401, 'URL inválida o expirada');
    }

    //...
})->name('unsubscribe');
```

```
// 0 con middleware
Route::get('/unsubscribe/{user}', function () {
    //...
})->middleware('signed')->name('unsubscribe');
```

Testing de rutas

```
// tests/Feature/RouteTest.php
use Tests\TestCase;

class RouteTest extends TestCase
{
    public function test_home_page_returns_successful_response()
    {
        $response = $this->get('/');

        $response->assertStatus(200);
    }

    public function test_authenticated_user_can_access_dashboard()
    {
        $user = User::factory()->create();

        $response = $this->actingAs($user)->get('/dashboard');

        $response->assertStatus(200);
    }

    public function test_guest_cannot_access_admin_routes()
    {
        $response = $this->get('/admin');

        $response->assertRedirect('/login');
    }
}
```

Mejores Prácticas

1. Organización de archivos de rutas

```
routes/
├─ web.php           # Rutas web públicas
├─ api.php           # API endpoints
├─ admin.php         # Panel de administración
├─ webhooks.php      # Webhooks de terceros
└─ console.php       # Comandos Artisan
```

2. Nomenclatura de rutas

```
// BIEN: Nombres descriptivos y consistentes
Route::get('/usuarios', [UserController::class, 'index'])
    ->name('usuarios.index');

Route::get('/usuarios/crear', [UserController::class, 'create'])
    ->name('usuarios.create');

// MAL: Nombres inconsistentes
Route::get('/usuarios', [UserController::class, 'index'])
    ->name('user_list');

Route::get('/usuarios/crear', [UserController::class, 'create'])
    ->name('createUser');
```

3. Usa controladores en lugar de closures

```
// MAL: Lógica en rutas
Route::get('/posts', function () {
    $posts = Post::with('author')
        ->where('published', true)
        ->orderBy('created_at', 'desc')
        ->paginate(15);

    return view('posts.index', compact('posts'));
});

// BIEN: Lógica en controlador
Route::get('/posts', [PostController::class, 'index']);
```

4. Agrupa rutas relacionadas

```
// BIEN: Rutas agrupadas lógicamente
Route::prefix('admin')
    ->middleware(['auth', 'admin'])
    ->name('admin.')
    ->group(function () {
        Route::resource('usuarios', UserController::class);
        Route::resource('posts', PostController::class);
        Route::resource('categorias', CategoryController::class);
    });
```

5. Usa Model Binding siempre que sea posible

```
// MAL: Búsqueda manual
Route::get('/usuarios/{id}', function ($id) {
    $user = User::findOrFail($id);
    return view('users.show', compact('user'));
});
```

```
// BIEN: Model Binding
Route::get('/usuarios/{user}', function (User $user) {
    return view('users.show', compact('user'));
});
```

6. Protege tus rutas adecuadamente

```
// Rutas públicas
Route::get('/', [HomeController::class, 'index']);

// Rutas autenticadas
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index']);
});

// Rutas de administrador
Route::middleware(['auth', 'admin'])->prefix('admin')->group(function () {
    Route::resource('users', UserController::class);
});
```

7. Usa rate limiting para proteger APIs

```
Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () {
    Route::apiResource('posts', PostController::class);
});
```

8. Documenta rutas complejas

```
/**
 * Endpoint para exportar reportes de ventas
 *
 * Requiere autenticación y rol de gerente
 * Rate limit: 10 peticiones por hora
 *
 * @QueryParam date_from Fecha inicio (YYYY-MM-DD)
 * @QueryParam date_to Fecha fin (YYYY-MM-DD)
 * @QueryParam format Formato de exportación (pdf|excel)
 */
Route::get('/reports/sales/export', [ReportController::class, 'exportSales'])
    ->middleware(['auth', 'role:manager', 'throttle:exports'])
    ->name('reports.sales.export');
```

Errores Comunes y Soluciones

Error 1: Route not found

Problema:

```
Target class [UserController] does not exist.
```

Solución:

```
// MAL: Falta el namespace completo
Route::get('/users', [UserController::class, 'index']);

// BIEN: Namespace completo
use App\Http\Controllers\UserController;
Route::get('/users', [UserController::class, 'index']);
```

Error 2: 404 con Model Binding

Problema: Ruta con model binding siempre devuelve 404

Solución: Verifica que el parámetro de ruta coincida con el nombre de la variable:

```
// MAL: No coinciden
Route::get('/posts/{id}', function (Post $post) {
    //...
});

// BIEN: Coinciden
Route::get('/posts/{post}', function (Post $post) {
    //...
});
```

Error 3: CSRF Token Mismatch**Problema:**

```
TokenMismatchException: CSRF token mismatch
```

Solución:

```
<!-- MAL: Falta token CSRF -->
<form method="POST" action="/profile">
    <!-- campos -->
</form>

<!-- BIEN: Incluye @csrf -->
<form method="POST" action="/profile">
    @csrf
    <!-- campos -->
</form>
```

Error 4: Rate Limit excedido inesperadamente

Problema: El rate limiting se activa demasiado pronto

Solución: Verifica que estés usando claves únicas para `by()` :

```
// MAL: Misma clave para diferentes límites
RateLimiter::for('api', function (Request $request) {
    return [
        Limit::perMinute(10)->by($request->user()->id),
        Limit::perHour(100)->by($request->user()->id), // Sobrescribe el primero
    ];
});

// BIEN: Claves únicas
RateLimiter::for('api', function (Request $request) {
    return [
        Limit::perMinute(10)->by('min:'.$request->user()->id),
        Limit::perHour(100)->by('hour:'.$request->user()->id),
    ];
});
```

Recursos Adicionales

- **Documentación Oficial:** <https://laravel.com/docs/12.x/routing>
- **Laracasts:** <https://laracasts.com/series/laravel-from-scratch>
- **API Reference:** <https://api.laravel.com/docs/12.x>